

OSNOVE GEOINFORMATIKE

Projektni zadatak

Katastar saobraćajne signalizacije

GIS aplikacija za evidenciju, prostornu analizu i AI detekciju
saobraćajnih znakova i semafora

Studenti	Danica Jovanović (RA116-2023) Uroš Popović (RA101-2023)
Predmet	Osnove geoinformatike
Odabrana tema	10 — Katastar saobraćajne signalizacije
Vrsta dokumenta	Tehnička dokumentacija za odbranu projekta

Sadržaj

1. Uvod	
1.1 Predmet i cilj projekta	
1.2 Odabrana tema i obrazloženje	
2. Arhitektura sistema	
2.1 Tehnologije	
2.2 Struktura projekta	
2.3 Tok podataka kroz sistem	
3. Deo 1 — Baza podataka i Python/SQL sloj	
3.1 Konekcija na bazu	
3.2 Model podataka	
3.3 Ručni unos podataka (INSERT)	
3.4 Učitavanje podataka u DataFrame	
3.5 CRUD operacije	
3.6 Upiti sa spajanjem tabela (JOIN)	
4. Deo 2 — Geoprostorna obrada podataka	
4.1 Izvor i priprema podataka	
4.2 Integracija sa PostGIS bazom	
4.3 Interaktivna GIS mapa	
4.4 Overlay analize	
4.5 Prostorni upiti	
5. Deo 3 — Mašinsko učenje i detekcija objekata	
5.1 Model i pristup treniranju	
5.2 Proces detekcije i geolociranje	
..... 5.3 Integracija detekcija sa katastrom	
5.4 Prikaz i ručna korekcija atributa	
5.5 Primeri prostornih analiza nad ML rezultatima	
6. Korisnički interfejs aplikacije	
7. Pokretanje aplikacije	
8. Zaključak	
8.1 Ostvareni rezultati	
8.2 Mogući pravci daljeg razvoja	
8.3 Zaključna ocena	
Literatura i izvori podataka	

1. Uvod

1.1 Predmet i cilj projekta

Projektni zadatak iz predmeta Osnove geoinformatike zahteva razvoj GIS aplikacije kroz tri celine koje se logički nadovezuju jedna na drugu: relacionu bazu podataka sa prostornim proširenjem, obradu i integraciju geoprostornih slojeva, i primenu mašinskog učenja za automatsku detekciju objekata od interesa. Cilj nije bio realizovati tri odvojena zadatka, već izgraditi jedinstven sistem u kojem svaki sledeći deo nadograđuje prethodni — baza iz prvog dela postaje odredište za geopodatke iz drugog dela, a rezultati modela iz trećeg dela se upisuju nazad u istu bazu i povezuju sa već evidentiranim objektima.

Rezultat je desktop/web aplikacija pokrenuta preko Streamlit okvira, sa PostgreSQL/PostGIS bazom kao centralnim skladištem podataka, koja korisniku omogućava pregled, unos, izmenu i prostornu analizu podataka o saobraćajnoj signalizaciji, kao i pokretanje AI detekcije nad fotografijama i automatsko ucrtavanje rezultata na mapu.

1.2 Odabrana tema i obrazloženje

Od ponuđenih tema u projektnom zadatku odabrana je tema **10 — Katastar saobraćajne signalizacije**, koja traži razvoj GIS aplikacije za upravljanje podacima o saobraćajnim znakovima, semaforima i signalizaciji, uz PostgreSQL/PostGIS bazu, prikaz slojeva na mapi, prostorne analize i algoritam mašinskog učenja za detekciju znakova sa fotografija ili video snimaka.

Tema je pogodna za grad Novi Sad jer OpenStreetMap/Geofabrik izvoz sadrži dovoljno atributski bogate podatke o putnoj mreži i raskrsnicama da se od njih može izgraditi realističan katastar, a istovremeno postoji jasna, praktično motivisana veza između tri dela zadatka: baza čuva podatke o saobraćajnoj signalizaciji, geoprostorni sloj omogućava njihov prikaz i analizu, a model mašinskog učenja služi za automatsko prepoznavanje saobraćajne signalizacije

2. Arhitektura sistema

2.1 Tehnologije

Aplikacioni i analitički deo sistema realizovan je u Python okruženju, dok se PostgreSQL/PostGIS koristi kao centralni relacioni i geoprostorni sloj za skladištenje podataka. Sistem je organizovan po funkcionalnim slojevima, pri čemu svaki sloj ima jasno definisanu odgovornost:

Sloj	Tehnologija	Uloga u sistemu
Baza podataka	PostgreSQL / PostGIS	Skladištenje relacionih i prostornih podataka
Pristup bazi	psycopg2	Direktno izvršavanje SQL/PostGIS upita
Obrada podataka	pandas	Tabelarni prikaz i transformacija podataka
Geoprostorna obrada	geopandas, shapely	Rad sa vektorskim slojevima i geometrijom

Sloj	Tehnologija	Uloga u sistemu
Mašinsko učenje	ultralytics (YOLO)	Detekcija saobraćajnih znakova na slikama
Interaktivna mapa	folium, streamlit-folium	Renderovanje slojeva i mape unutar aplikacije
Korisnički interfejs	Streamlit	Web aplikacija sa više funkcionalnih stranica
Konfiguracija	python-dotenv	Učitavanje kredencijala baze iz .env fajla

2.2 Struktura projekta

Kod je organizovan u jasno odvojene module, pri čemu svaki modul odgovara jednom delu projektnog zadatka ili jednoj tehničkoj odgovornosti:

projekat/	
├ main.py	# Ulazna tačka – pokreće Streamlit aplikaciju
├ src/	
│ └ database/	
│ │ └ connection.py	# psycopg2 konekcija (env promenljive)
│ │ └ create_tables.py	# DDL – kreiranje 5 tabela
│ │ └ seed_manual_data.py	# Ručni INSERT (min. 5 redova po tabeli)
│ │ └ insert_osm_to_tables.py	# Masovni unos OSM/Geofabrik podataka
│ │ └ crud.py	# CREATE / READ / UPDATE / DELETE
│ │ └ queries.py	# 10 upita sa JOIN-om
│ └ geo/	
│ │ └ load_data.py	# Učitavanje .shp slojeva (geopandas)
│ │ └ preprocessing.py	# Filtriranje na područje Novog Sada
│ │ └ overlay_analysis.py	# clip / buffer / intersection / union / difference
│ │ └ spatial_analysis.py	# nearest / within / intersects, heatmapa
│ │ └ map_view.py	# Generisanje samostalne HTML mape
│ └ ml/	
│ │ └ detector.py	# YOLO detekcija, upis rezultata u bazu
│ └ app/	
│ │ └ streamlit_app.py	# Kompletan korisnički interfejs (7 stranica)
└ models/	
└ best.pt	# Sopstveno istreniran YOLO model
└ results/	
│ └ analysis/	# CSV izveštaji overlay i prostornih analiza
│ └ maps/	# Izvezene HTML mape (npr. heatmapa)
│ └ ml/detections/	# Anotirane slike sa rezultatima detekcije

Slika 2.1 — Struktura direktorijuma projekta, podeljena po funkcionalnim celinama.

2.3 Tok podataka kroz sistem

Podaci kroz sistem prolaze kroz tri međusobno povezane faze koje odgovaraju glavnim delovima projektnog zadatka. Sve faze koriste zajedničku PostgreSQL/PostGIS bazu, koja predstavlja centralno mesto za čuvanje atributskih i prostornih podataka:

1. **Deo 1 (SQL):** kreira se osnovna relacionala i prostorna šema sa tabelama ulice, raskrsnice, saobraćajni_znakovi, semafori i ml_detekcije. Tabele su povezane stranim ključevima, a ručno pripremljeni podaci koriste se za demonstraciju INSERT, CRUD i JOIN operacija pre učitavanja realnih prostornih podataka.

2. **Deo 2 (GEO):** originalni Geofabrik shapefile slojevi učitavaju se pomoću biblioteke GeoPandas, filtriraju na studijski obuhvat Novog Sada i čuvaju kao obrađeni GeoJSON slojevi. U demonstracionom toku masovnog unosa prethodni podaci se brišu, a zatim se u iste PostGIS tabele unose realni OSM podaci. Nakon unosa izvršavaju se povezivanje znakova sa obližnjim ulicama, kreiranje prostornih indeksa i različite prostorne i overlay analize.
3. **Deo 3 (ML):** korisnik učitava fotografiju i unosi približne koordinate mesta na kojem je fotografija napravljena. Fino podešeni YOLOv8s model detektuje saobraćajne znakove i za svaki objekat vraća klasu, pouzdanost i bounding box. Rezultati se upisuju u tabelu `ml_detekcije`, čuvaju kao anotirana fotografija i pokušavaju da se povežu sa najbližim postojećim znakom u radijusu od 50 metara.

3. Deo 1 — Baza podataka i Python/SQL sloj

Ovaj deo obuhvata uspostavljanje PostgreSQL/PostGIS baze, definisanje relacionog i prostornog modela podataka, ručni i masovni unos podataka, kao i izvršavanje CRUD i analitičkih SQL operacija iz Python koda. Za direktnu komunikaciju sa bazom koristi se biblioteka `psycopg2`, bez ORM sloja, dok se rezultati SQL upita učitavaju u `pandas` i `GeoPandas` strukture radi tabelarnog prikaza i geoprostorne obrade. Parametrizovani SQL upiti koriste se radi bezbednog prosleđivanja korisničkih vrednosti.

3.1 Konekcija na bazu

Parametri konekcije — naziv baze, korisnik, lozinka, host i port — čuvaju se u lokalnom `.env` fajlu i učitavaju pomoću biblioteke `python-dotenv`. Time se kredencijali ne hardkoduju u izvorni kod niti čuvaju u Git repozitorijumu.

Pored osnovne funkcije `get_connection()`, implementirani su kontekstni menadžeri `managed_connection()` i `managed_cursor()`. Oni garantuju da će uspešna transakcija biti potvrđena operacijom `commit`, neuspešna vraćena pomoću `rollback`, a konekcija i kursor zatvoreni čak i kada nastane izuzetak.

```
def get_connection():
    return psycopg2.connect(
        dbname=os.environ.get(
            "DB_NAME",
            "katastar_saobracajne_signalizacije",
        ),
        user=os.environ.get("DB_USER", "postgres"),
        password=os.environ["DB_PASSWORD"],
        host=os.environ.get("DB_HOST", "localhost"),
        port=os.environ.get("DB_PORT", "5432"),
    )

@contextmanager
def managed_connection():
    conn = get_connection()
```

```

try:
    yield conn
    conn.commit()
except Exception:
    conn.rollback()
    raise
finally:
    conn.close()

@contextmanager
def managed_cursor():
    with managed_connection() as conn:
        with conn.cursor() as cur:
            yield cur

```

Isečak 3.1 — `src/database/connection.py`

3.2 Model podataka

Baza sadrži pet tabela koje pokrivaju sve entitete relevantne za katastar saobraćajne signalizacije. Tabele su povezane stranim ključevima tako da odražavaju stvarnu hijerarhiju: ulica sadrži znakove, raskrsnica sadrži semafore, a ML detekcija se (kada je moguće) povezuje sa postojećim znakom.

Tabela	Ključne kolone	Geometrija	Veza (FK)
ulice	naziv, tip_ulice, ogranicenje_brzine, broj_traka, duzina_km	LineString / MultiLineString	—
raskrsnice	naziv, tip, broj_prilaza, ima_semafor, prometnost	Point	—
saobracajni_znakovi	tip_znaka, opis, stanje, datum_postavljanja, proizvođač	Point	ulica_id → ulice.id
semafori	status, tip, datum_servisa, broj_signalnih_glava	Point	raskrsnica_id → raskrsnice.id
ml_detekcije	klasa, confidence, naziv_slike, datum, model, bbox_x1..y2	Point	znak_id → saobracajni_znakovi.id

Sve geometrijske kolone koriste koordinatni referentni sistem EPSG:4326, koji je standardan za GPS i OpenStreetMap podatke. Tačkasti entiteti čuvaju se kao Point, dok se geometrija ulica nakon pripreme za OSM unos čuva kao MultiLineString. Za analize rastojanja i bafera podaci se privremeno transformišu u projektovani sistem EPSG:32634, u kome su jedinice izražene u metrima.

Tabele su povezane stranim ključevima tako da jedna ulica može imati više saobraćajnih znakova, jedna raskrsnica više semafora, dok se ML detekcija, kada je moguće, povezuje sa postojećim znakom. Za veze ulica–znak i raskrsnica–semafor koristi se ON DELETE CASCADE, dok se kod veze ML detekcija–znak koristi ON DELETE SET NULL, kako bi istorija automatskih detekcija ostala sačuvana i nakon brisanja povezanog znaka.

Primer definicije tabele

```
CREATE TABLE IF NOT EXISTS saobracajni_znakovi (
    id SERIAL PRIMARY KEY,
    tip_znaka VARCHAR(100) NOT NULL,
    opis TEXT,
    stanje VARCHAR(50),
    datum_postavljanja DATE,
    proizvođač VARCHAR(100),
    ulica_id INTEGER REFERENCES ulice(id) ON DELETE CASCADE,
    geom geometry(Point, 4326)
);
```

Isečak 3.2 — *src/database/create_tables.py*

3.3 Ručni unos podataka (INSERT)

U skladu sa zahtevom da se u svaku tabelu ručno unese najmanje pet redova, modul `seed_manual_data.py` sadrži eksplicitne INSERT naredbe sa realnim nazivima novosadskih ulica i raskrsnica (Bulevar Mihajla Pupina, Bulevar Oslobođenja, Futoška ulica, kružni tok na Limanu itd.), čime se demonstrira direktan unos pre nego što baza kasnije biva napunjena masovnim OSM podacima.

```
cur.execute("""
    INSERT INTO raskrsnice (naziv, tip, broj_prilaza, ima_semafor, prometnost, geom)
    VALUES (%s, %s, %s, %s, %s, ST_SetSRID(ST_MakePoint(%s, %s), 4326))
    RETURNING id;
""", (naziv, tip, broj_prilaza, ima_semafor, prometnost, lon, lat))
```

Isečak 3.3 — *geometrija se pri unosu konstruiše direktno u SQL-u preko ST_MakePoint / ST_SetSRID.*

3.4 Učitavanje podataka u DataFrame

Čitanje tabela u pandas DataFrame realizovano je generičkom funkcijom koja prima naziv tabele i vraća prvih 100 redova sortiranih po id-ju, što se koristi i u CRUD demonstraciji i u stranici „Tabele“ unutar aplikacije:

```
ALLOWED_TABLES = {
    "ulice",
    "raskrsnice",
    "saobracajni_znakovi",
    "semafori",
    "ml_detekcije",
}

def read_table(table_name):
    if table_name not in ALLOWED_TABLES:
        raise ValueError(
            f"Nedozvoljen naziv tabele: {table_name}"
        )

    query = f"""
        SELECT *
        FROM {table_name}
        ORDER BY id
        LIMIT 100;
    """
```

```
with managed_connection() as conn:
    return pd.read_sql_query(query, conn)
    return pd.read_sql_query(query, conn)
```

3.5 CRUD operacije

Sve četiri operacije su implementirane za dva ključna entiteta — saobraćajne znakove i semafore — sve promenljive vrednosti prosleđuju se kroz parametrizovane SQL upite, dok se transakcije izvršavaju pomoću `managed_cursor()` kontekstnog menadžera

- **Create** — prilikom dodavanja novog znaka, njegova `ulica_id` se ne unosi ručno već sistem zatim pokušava da pronađe najbližu ulicu u maksimalnom radijusu od 100 metara. Funkcija `ST_DWithin` prvo izdvađa ulice koje se nalaze u dozvoljenom radijusu, dok `ST_Distance` bira najbližu među njima. Geometrije se privremeno tretiraju kao tip `geography`, zbog čega se rastojanje izražava u metrima. Ako u radijusu od 100 metara ne postoji nijedna ulica, znak se ipak upisuje u bazu, ali njegov `ulica_id` ostaje `NULL`.
- **Read** — `read_table()` vraća stanje tabele kao `DataFrame`, spreman za prikaz u interfejsu.
- **Update** — izmena stanja znaka (dobro / oštećen / potrebno održavanje) ili statusa semafora, po id-ju.
- **Delete** — brisanje po id-ju, uz oslanjanje na `ON DELETE` politiku `FK` za posledične promene u zavisnim tabelama.

```
SELECT u.id
FROM ulice u
WHERE u.geom IS NOT NULL
      AND ST_DWithin(
          u.geom::geography,
          ST_SetSRID(
              ST_MakePoint(%s, %s),
              4326
          )::geography,
          %s
      )
ORDER BY ST_Distance(
    u.geom::geography,
    ST_SetSRID(
        ST_MakePoint(%s, %s),
        4326
    )::geography
)
LIMIT 1
```

Isečak 3.4 — `src/database/crud.py` — automatsko povezivanje novog znaka sa najbližom ulicom.

3.6 Upiti sa spajanjem tabela (JOIN)

Modul `queries.py` sadrži deset upita, umesto traženog minimuma od pet, sa namerom da pokriju različite tipove spajanja i agregacija: unutrašnji i levi `JOIN`, anti-join (pronalaženje redova bez para), grupisanje sa `COUNT/AVG` i filtriranje po više uslova istovremeno (klauzula `WHERE`).

#	Naziv upita	Tip spajanja / tehnika
1	Ulice sa ograničenjem > 50 km/h i broj znakova na njima	LEFT JOIN + GROUP BY + COUNT
2	Znakovi koji nisu u dobrom stanju, sa nazivom ulice	INNER JOIN + WHERE
3	Aktivni semafori sa nazivom raskrsnice	INNER JOIN + WHERE
4	Semafori na kontrolisanim raskrsnicama	INNER JOIN + WHERE + LIMIT
5	ML detekcije povezane sa znacima (confidence > 0.5)	INNER JOIN + WHERE
6	Deset najdužih ulica i broj znakova na njima	LEFT JOIN + GROUP BY + ORDER BY
7	Prosečna dužina ulica po kategoriji	GROUP BY + AVG (bez JOIN-a)
8	Glavne ulice bez ijednog evidentiranog znaka	LEFT JOIN (anti-join, IS NULL)
9	Broj znakova po stanju	GROUP BY + COUNT
10	ML detekcije visoke pouzdanosti i status povezanosti	LEFT JOIN + CASE WHEN

Primer upita 8 — anti-join tehnika

Upit koji izdvaja glavne ulice (primary/secondary/tertiary/trunk/motorway) bez ijednog evidentiranog znaka posebno je koristan sa stanovišta katastra jer identifikuje potencijalne propuste u terenskoj evidenciji:

```
SELECT
    u.id,
    u.naziv,
    u.tip_ulice,
    ROUND(u.duzina_km::numeric, 3) AS duzina_km
FROM ulice u
LEFT JOIN saobracajni_znakovi z
    ON z.ulica_id = u.id
WHERE z.id IS NULL
    AND u.tip_ulice IN (
        'primary',
        'secondary',
        'tertiary',
        'trunk',
        'motorway'
    )
ORDER BY u.duzina_km DESC
LIMIT 20;
```

Isečak 3.5 — `src/database/queries.py, query_8_streets_without_signs()`

Napomena

Svih deset upita se izvršava i preko Streamlit stranice „SQL upiti“, gde se rezultat prikazuje kao tabela i može preuzeti u CSV formatu

4. Deo 2 — Geoprostorna obrada podataka

Drugi deo projekta integriše realne geoprostorne podatke sa bazom izgrađenom u Delu 1, i nad njima sprovodi analitičke operacije karakteristične za GIS — overlay tehnike i prostorne upite. Ovaj deo direktno koristi geopandas kao most između shapefile formata i PostGIS baze.

4.1 Izvor i priprema podataka

Kao izvor vektorskih podataka korišćen je **Geofabrik OSM izvoz** za područje Srbije, iz kog su preuzeti slojevi puteva (`gis_osm_roads_free_1.shp`), saobraćajnih objekata (`gis_osm_traffic_free_1.shp`) i tačaka od interesa (`gis_osm_pois_free_1.shp`). Slojevi se učitavaju preko `geopandas.read_file()` i zatim filtriraju na područje Novog Sada pomoću geometrijskog isecanja (`clip`) po graničnom pravougaoniku:

```
NOVI_SAD_BBOX = box(  
    19.70,  
    45.18,  
    19.98,  
    45.35,  
)  
  
def filter_by_novi_sad_area(gdf):  
    novi_sad_area = gpd.GeoDataFrame(  
        geometry=[NOVI_SAD_BBOX],  
        crs="EPSG:4326",  
    )  
  
    return gpd.clip(gdf, novi_sad_area)
```

Isečak 4.1 — `src/geo/preprocessing.py`

Nakon isecanja, slojevi se dodatno filtriraju po tematskoj klasi (`fclass`) — za puteve se zadržavaju samo relevantne kategorije (`motorway`, `primary`, `secondary`, `residential...`), za saobraćajne objekte samo znakovi i semafori, a za POI samo kategorije bitne za saobraćajnu bezbednost (`škole`, `bolnice`, `autobuske stanice`). Rezultat se čuva kao GeoJSON u `data/processed/`, čime priprema podataka postaje reproducibilan, jednom pokrenut korak, odvojen od učitavanja u bazu.

4.2 Integracija sa PostGIS bazom

Pripremljeni GeoJSON slojevi se zatim masovno učitavaju u iste tabele iz Dela 1 (`insert_osm_to_tables.py`), čime se ručno uneti demonstracioni podaci zamenjuju stvarnim podacima za Novi Sad. Pre unosa, baza se priprema dodatnim koracima koji nisu bili neophodni za ručni unos, ali postaju bitni pri radu sa hiljadama geometrija:

- **Kolona `osm_id`** (`BIGINT UNIQUE`) dodaje se u sve tabele ulice, raskrsnice, saobraćajni znakovi i semafori kako bi ponovljeno pokretanje skripte bilo idempotentno — `INSERT` koristi `ON CONFLICT (osm_id) DO NOTHING`.
- GiST prostorni indeksi kreiraju se nad geometrijama navedene četiri OSM tabele.
- **Tip geometrije ulica** se menja u `MultiLineString (ST_Multi)`, jer OSM putna mreža prirodno sadrži deonice sa više segmenata.

- **Automatsko povezivanje** — nakon unosa, svaki znak bez ulica_id se preko istog KNN principa (ORDER BY geom <-> ...) povezuje sa geometrijski najbližom ulicom, u jednom UPDATE upitu nad celom tabelom.

```
UPDATE saobracajni_znakovi z
SET ulica_id = (
    SELECT u.id
    FROM ulice u
    ORDER BY u.geom <-> z.geom
    LIMIT 1
)
WHERE z.ulica_id IS NULL;
```

Isečak 4.2 — `src/database/insert_osm_to_tables.py, link_signs_to_nearest_street()`

4.3 Interaktivna GIS mapa

Centralna funkcionalnost drugog dela je interaktivna mapa (Streamlit stranica „Mapa“), izgrađena kombinacijom folium i streamlit-folium. Podaci se za prikaz učitavaju direktno iz PostGIS baze preko `geopandas.read_postgis()`, a ne iz statičkih fajlova, čime mapa uvek prikazuje trenutno stanje baze — uključujući i najsvežije ML detekcije.

Funkcionalnost	Realizacija
Uključivanje/isključivanje slojeva	Checkbox kontrole za Ulice, Znakove, Semafore i ML detekcije; folium.LayerControl za dodatno upravljanje unutar same mape
Grupisanje markera	MarkerCluster za znakove, semafore i ML detekcije radi čitljivosti pri gušćim skupovima tačaka
Popup / tooltip informacije	Svaki marker prikazuje atribut iz baze (tip, opis, stanje, confidence...) pri klikom/hoveru
Izbor osnove (raster podloga)	Padajući meni za izbor tile provajdera kao raster podloge ispod vektorskih slojeva
Simbologija slojeva	Zaseban panel u kom se nezavisno bira boja i stil za svaki sloj — vidi niže
Podešavanje pogleda	Unos centralnih koordinata i početnog zuma
Uprošćavanje geometrije	ST_SimplifyPreserveTopology primenjen na sloju ulica radi bržeg renderovanja

Vektorski slojevi se prikazuju preko raster tile osnove (OpenStreetMap i alternativni stilovi), u skladu sa zahtevom da se vektorski podaci prikazuju preko raster podloge.

Pored uključivanja i isključivanja slojeva, korisniku je dostupan i poseban panel „Podešavanja simbologije“, odvojen od osnovnih kontrola mape, u kom se stil svakog sloja menja nezavisno od ostalih. Rešenje prati jednu tehničku razliku između linijskih i tačkastih slojeva: folium.GeoJson prihvata proizvoljnu boju preko `style_function`, pa se za sloj ulica nudi pravi `color_picker` sa slobodnim izborom boje, debljine i providnosti linije; folium.Icon, koji se koristi za markere, ograničen je na fiksnu paletu imenovanih boja, pa se za znakove, semafore i ML detekcije boja bira preko padajućeg menija nad tom paletom, uz dodatni izbor ikonice.

```
roads_color = st.color_picker("Boja linije", "#38bdf8")
```

```
roads_weight = st.slider("Debljina linije", 1.0, 8.0, 2.4, 0.2)
roads_opacity = st.slider("Providnost linije", 0.1, 1.0, 0.66, 0.05)

folium.GeoJson(
    roads,
    style_function=lambda _feature,
    _c=roads_color,
    _w=roads_weight,
    _o=roads_opacity: {
        "color": _c,
        "weight": _w,
        "opacity": _o,
    },
).add_to(m)
```

Isečak 4.3 — `src/app/streamlit_app.py, map_page()`; izabrane vrednosti se prosleđuju direktno u `style_function`.

Za znak tipa „stop“ zadržana je opcija automatskog isticanja crvenom bojom nezavisno od izabrane osnovne boje sloja znakova — ne kao ograničenje, već kao svesna kartografska odluka, jer taj znak nosi bezbednosni prioritet koji opravdava vizuelno izdvajanje. Opcija se može isključiti checkbox-om u istom panelu, čime korisnik zadržava punu kontrolu nad izgledom mape.

4.4 Overlay analize

Nad geometrijom iz baze sprovedeno je pet overlay tehnika, čime je pokriven ceo skup primera naveden u zadatku (clip, union, intersection, buffer). Sve analize rade u projektovanom metričkom sistemu EPSG:32634 (UTM zona 34N), koji za Vojvodinu daje rastojanja u metrima sa zanemarljivim izobličenjem — što je preduslov za smislene bafere i merenja.

#	Tehnika	Opis i primenjeni parametri
1	Clip	Isecanje ulica prema konveksnom omotaču (convex hull) svih ulica, uvećanom baferom od 500 m — definiše „zonu analize“.
2	Buffer	Bafer od 100 m oko glavnih puteva (primary/secondary/tertiary/trunk/motorway).
3	Intersection	Preseci znakova sa baferskom zonom glavnih puteva (spatial join, predicate=within).
4	Union	Spajanje baferskih zona znakova (30 m) i semafora (50 m) u jedinstvenu „zonu pokrivenosti signalizacijom“.
5	Difference	Delovi putne mreže koji ostaju izvan zone pokrivenosti signalizacijom — kandidati za proveru na terenu.

```
def overlay_5_difference_roads_outside_signalization_zone():
    ulice_m = ulice.to_crs(epsg=32634)
    union_zone = overlay_4_union_signalization_zones()
    roads_union = gpd.GeoDataFrame(
        [{"naziv": "sve_ulice", "geom": ulice_m.union_all()}],
        geometry="geom", crs=ulice_m.crs
    )
    return gpd.overlay(roads_union, union_zone, how="difference")
```

Isečak 4.3 — `src/geo/overlay_analysis.py` — svaka analiza upisuje rezultat u `results/analysis/*.csv`

4.5 Prostorni upiti

Pored overlay tehnika, implementirano je šest prostornih analiza koje kombinuju različite prostorne predikate (within, intersects) i sjoin_nearest, uz jednu vizuelnu analizu (heatmapa gustine signalizacije):

1. **Najbliža ulica za svaki znak** — gpd.sjoin_nearest, sa izračunatim rastojanjem u metrima (udaljenost_m).
2. **Znakovi u blizini semafora** — bafer od 50 m oko semafora + sjoin sa predikatom within.
3. **Broj znakova po tipu** — atributska agregacija nad prostornim slojem.
4. **Semafori u blizini raskrsnica** — bafer od 30 m oko raskrsnica + sjoin within, kao provera konzistentnosti podataka.
5. **ML detekcije naspram katastra** — sjoin sa predikatom intersects (za razliku od within u prethodnim analizama) unutar 50 m bafera oko postojećih znakova; detekcije bez poklapanja izdvajaju se kao kandidati za nove, još neevidentirane znakove.
6. **Heatmapa gustine signalizacije** — folium.plugins.HeatMap nad kombinovanim koordinatama znakova i semafora, izvezena kao samostalna HTML mapa u results/maps/.

Zašto within na jednom mestu, a intersects na drugom

Predikat within je korišćen kada tačka mora biti strogo unutar zone (npr. znak unutar bafera puta). Predikat intersects je namerno korišćen za poređenje ML detekcija sa katastrom jer dozvoljava i granični slučaj poklapanja (detekcija na ivici bafera), što je konzervativnija provera pogodnija za identifikaciju kandidata za dopunu katastra.

5. Deo 3 — Mašinsko učenje i detekcija objekata

Treći deo projekta uvodi automatsku detekciju saobraćajnih znakova sa fotografija i povezuje rezultate mašinskog učenja sa postojećim PostGIS katastrom.

Korisnik kroz Streamlit aplikaciju učitava fotografiju, unosi približnu geografsku lokaciju mesta snimanja i bira minimalni prag pouzdanosti. YOLO model zatim pronalazi objekte na slici, određuje njihovu klasu, pouzdanost i položaj unutar fotografije, nakon čega se rezultati čuvaju u tabeli ml_detekcije.

Na ovaj način ML deo nije izdvojen od ostatka sistema, već koristi istu bazu, istu mapu i iste prostorne analize kao i ručno i OSM poreklo podataka.

5.1 Model i pristup treniranju

Za detekciju je korišćen **YOLO model (ultralytics)**, kao početna osnova korišćene su težine modela yolov8s.pt, prethodno obučene na COCO skupu podataka, nakon čega je izvršeno fino podešavanje na DFG Traffic Sign Dataset-u. Ovakav postupak predstavlja transfer learning i omogućava da model iskoristi prethodno naučene vizuelne karakteristike, a zatim ih prilagodi specifičnom zadatku prepoznavanja saobraćajne signalizacije.

```

ROOT = Path(__file__).resolve().parents[2]

RESULTS_DIR = ROOT / "results" / "ml" / "detections"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

CUSTOM_MODEL_PATH = ROOT / "models" / "best.pt"

def get_model():
    if not CUSTOM_MODEL_PATH.exists():
        raise FileNotFoundError(
            f"Model nije pronađen na putanji: {CUSTOM_MODEL_PATH}"
        )

    print("Koristi se model fino podešen za detekciju saobraćajnih znakova.")
    return YOLO(str(CUSTOM_MODEL_PATH))

```

Isečak 5.1 — *src/ml/detector.py*

5.2 Proces detekcije i geolociranje

Korisnik kroz aplikaciju učitava fotografiju i unosi njenu približnu geografsku lokaciju (širinu i dužinu) — koordinate mesta gde je snimak napravljen. Model zatim vraća bounding box-ove, klase i pouzdanost (confidence) za svaki detektovani objekat na slici. Kako jedna fotografija često sadrži više znakova na istoj lokaciji, uveden je mehanizam kozmetičkog razmicanja tačaka:

```

def apply_jitter(lon, lat, index, total, radius_m=4.0):
    """
    Kozmetički (ne stvarni geolokacioni) pomak: kad je na jednoj fotografiji
    detektovano više znakova, raspoređuje ih ravnomerno po malom krugu oko
    unete tačke da se markeri na mapi ne preklapaju tačka-na-tačku.
    """
    if total <= 1:
        return lon, lat
    angle = 2 * math.pi * index / total
    lat_offset = (radius_m * math.cos(angle)) / 111_320
    lon_offset = (radius_m * math.sin(angle)) / (111_320 * math.cos(math.radians(lat)))
    return lon + lon_offset, lat + lat_offset

```

Isečak 5.2 — *bez ove korekcije bi svi znakovi sa iste fotografije dobili identičnu tačku, pa bi se na mapi preklapali.*

Ovaj pomak ne predstavlja nezavisno određenu geografsku lokaciju svakog znaka. Sve detekcije potiču od iste približne lokacije fotografije, dok se male pomerene koordinate koriste radi njihovog prostornog razdvajanja. U trenutnoj implementaciji upravo te pomerene koordinate čuvaju se i u tabeli `ml_detekcije`.

Za svaku detekciju iznad zadatog praga pouzdanosti (podrazumevano 0,25, podesivo u interfejsu) u bazu se upisuju: klasa, confidence, naziv slike, datum, koordinate tačke, naziv modela i koordinate bounding box-a (bbox_x1..y2), radi potpune sledljivosti odluke modela.

5.3 Integracija detekcija sa katastrom

Ključni deo integracije je automatsko povezivanje svake nove ML detekcije sa postojećim znakom iz katastra, ukoliko takav znak već postoji u neposrednoj blizini. Ovo se radi preko PostGIS funkcije `ST_DistanceSphere`, koja računa rastojanje na sferi (u metrima) direktno iz geografskih koordinata:

```
def find_nearest_sign(cur, lon, lat, max_distance_m=50):
    cur.execute("""
        SELECT z.id, ST_DistanceSphere(z.geom,
            ST_SetSRID(ST_MakePoint(%s, %s), 4326)) AS udaljenost_m
        FROM saobracajni_znakovi z
        WHERE z.geom IS NOT NULL
        ORDER BY udaljenost_m
        LIMIT 1;
    """, (lon, lat))
    result = cur.fetchone()
    if result is not None and result[1] <= max_distance_m:
        return result[0]
    return None
```

Isečak 5.3 — ako se u krugu od 50 m ne pronađe postojeći znak, detekcija ostaje bez znak_id (NULL) i tretira se kao potencijalno nov, još neevidentiran znak.

Ova granica od 50 metara nije proizvoljna — usklađena je sa istom vrednošću korišćenom u prostornoj analizi iz poglavlja 4.5 (ML detekcije naspram katastra), tako da se rezultati detekcije u realnom vremenu i naknadna grupna analiza oslanjaju na istu definiciju „poklapanja“.

5.4 Prikaz i ručna korekcija atributa

Sve detekcije se automatski prikazuju kao poseban sloj na interaktivnoj mapi (poglavlje 4.3), grupisane u MarkerCluster, sa popup prozorom koji prikazuje klasu, pouzdanost, naziv slike i datum. Pored automatskog prikaza, aplikacija ispunjava zahtev za mogućnost izmene atributa kroz posebnu formu — stranica „AI detekcija“ ima tab „Korekcija zapisa“, u kojoj operater može, nakon stručne provere, ručno izmeniti klasu, confidence ili opis zapisa po id-ju:

```
cur.execute("""
    UPDATE ml_detekcije
    SET klasa = %s, confidence = %s, opis = %s
    WHERE id = %s;
""", (nova_klasa, novi_conf, novi_opis, det_id))
```

Isečak 5.4 — src/app/streamlit_app.py, tab_correct unutar ml_page()

5.5 Primeri prostornih analiza nad ML rezultatima

Kao što je opisano u poglavlju 4.5, analiza ml_detekcije_vs_katastar dodatno obrađuje rezultate detekcije: za svaku detekciju proverava da li upada u baferovanu zonu (50 m) oko postojećeg znaka istog ili sličnog tipa, i eksplicitno označava red kolonom poklapa_se_sa_katastrom (True/False). Ovim se dobija spisak koji direktno odgovara na pitanje koje je za katastar praktično najbitnije: koje AI detekcije predstavljaju već poznate znakove, a koje ukazuju na znakove koji još nisu uneti u zvaničnu evidenciju.

Praktični značaj

Ovakav pristup pretvara model iz čiste demonstracije prepoznavanja slike u alat za održavanje katastra — razlika između „detektovano i potvrđeno“ i „detektovano, a nepotvrđeno“ jeste upravo ono što bi realni sistem za upravljanje saobraćajnom signalizacijom morao da prati.

6. Korisnički interfejs aplikacije

Glavne funkcionalnosti sistema objedinjene su u jedinstvenoj Streamlit web aplikaciji sa sedam funkcionalnih stranica. Kroz interfejs je moguće pregledati trenutno stanje baze, koristiti interaktivnu mapu, izvršavati CRUD operacije nad saobraćajnim znakovima, pokretati pripremljene SQL upite, pregledati rezultate prostornih analiza i izvršavati AI detekciju. Priprema Geofabrik podataka i generisanje CSV rezultata prostornih i overlay analiza izvršavaju se prethodno kroz odgovarajuće Python module, dok aplikacija omogućava njihov pregled i preuzimanje.

Stranica	Sadržaj
Dashboard	Pregled broja redova u svih pet tabela, poslednjih ML detekcija, raspodele znakova i kategorija puteva, kao i statusa konekcije sa bazom i dostupnosti YOLO modela.
Mapa	Interaktivna GIS mapa sa slojevima ulica, saobraćajnih znakova, semafora i ML detekcija. Omogućeni su uključivanje i isključivanje slojeva, izbor raster podloge, grupisanje markera, podešavanje centra i nivoa uvećanja, fullscreen prikaz i promena simbologije slojeva.
Tabele	Tabelarni prikaz svih pet tabela iz PostgreSQL/PostGIS baze, uz izbor broja prikazanih redova, tekstualnu pretragu kroz rezultate i preuzimanje prikazanih podataka u CSV formatu.
CRUD	Forme za dodavanje, izmenu stanja i brisanje saobraćajnih znakova, organizovane u tri taba. Pri dodavanju korisnik unosi opisne atribute i geografsku lokaciju znaka, dok se povezivanje sa obližnjom ulicom izvršava u Python/SQL sloju.
SQL upiti	Izbor i pokretanje deset pripremljenih analitičkih SQL upita, od kojih osam koristi JOIN operacije, uz tabelarni prikaz rezultata i mogućnost preuzimanja u CSV formatu.
Analize	Pregled prethodno generisanih rezultata prostornih i overlay analiza. Korisnik bira dostupni CSV rezultat, pregleda njegove atribute, broj redova i kolona i može da ga preuzme. Same analize pokreću se prethodno kroz module <code>spatial_analysis.py</code> i <code>overlay_analysis.py</code> .
AI detekcija	Učitavanje fotografije, unos približnih koordinata mesta snimanja i podešavanje minimalnog praga pouzdanosti. Stranica prikazuje anotiranu fotografiju i tabelu detekcija, čuva rezultate u PostGIS bazi, omogućava pregled i filtriranje istorije i ručnu korekciju klase, confidence vrednosti i opisa zapisa.

Navigacija između stranica realizovana je kroz bočni meni, koji dodatno prikazuje status baze, dostupnost modela i broj podržanih klasa. Korisnički interfejs koristi prilagođenu tamnu temu, kartice, ikonice, statusne indikatore i responzivni raspored elemenata, čime se različiti delovi sistema objedinjuju u vizuelno konzistentnu aplikaciju.

7. Pokretanje aplikacije

Aplikacija se pokreće preko jedinstvene ulazne tačke `main.py`, koja u pozadini poziva Streamlit CLI nad glavnim UI modulom:

```
def main():
```



```
subprocess.run([
    sys.executable,
    "-m",
    "streamlit",
    "run",
    "src/app/streamlit_app.py"
])
```

Isečak 7.1 — *main.py*

Preduslovi za pokretanje:

1. Podignut PostgreSQL server sa instaliranim PostGIS proširenjem.
2. Kreirana .env datoteka sa parametrima konekcije (DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT).
3. Instalirane Python zavisnosti (psycopg2, pandas, geopandas, shapely, folium, streamlit, streamlit-folium, ultralytics, python-dotenv).
4. Redom pokrenuti: create_tables.py → seed_manual_data.py → preprocessing.py → insert_osm_to_tables.py → overlay_analysis.py → spatial_analysis.py
5. Pokrenuti python main.py — aplikacija se otvara na lokalnoj adresi u pregledaču.

8. Zaključak

8.1 Ostvareni rezultati

Implementacijom projekta objedinjena su tri glavna funkcionalna dela: relaciona PostgreSQL/PostGIS baza podataka, geoprostorna obrada i vizuelizacija podataka, kao i detekcija saobraćajnih znakova pomoću YOLO modela. Sistem omogućava ručni i masovni unos podataka, izvršavanje CRUD i analitičkih SQL operacija, prostorne analize, prikaz podataka na interaktivnoj mapi i čuvanje rezultata ML detekcija kao prostornih zapisa u istoj bazi. Korisnički interfejs povezuje ove funkcionalnosti i omogućava pregled baze, pokretanje SQL upita, rad sa saobraćajnim znakovima, pregled rezultata prostornih analiza i izvršavanje AI detekcije nad fotografijama.

8.2 Mogući pravci daljeg razvoja

Sistem u sadašnjem obliku ispunjava sve zahteve projektnog zadatka, uključujući i podešavanje simbologije po sloju. Kao prirodan sledeći korak, van okvira samog zadatka, mogu se razmotriti sledeća proširenja:

- Uvođenje preciznijeg geolociranja znakova korišćenjem GPS i EXIF podataka fotografije, umesto oslanjanja samo na ručno unetu približnu lokaciju mesta snimanja.
- Povezivanje ML detekcije sa postojećim znakom na osnovu prostorne blizine i poklapanja klase, čime bi se smanjila mogućnost pogrešnog povezivanja različitih tipova znakova.
- Verzionisanje promena atributa (istorija izmena stanja znaka/semafora), korisno za praćenje održavanja opreme kroz vreme.
- Čuvanje izabrane simbologije po korisniku (npr. u lokalnoj konfiguraciji), tako da se podešen izgled slojeva pamti između sesija.

8.3 Zaključna ocena

Sistem predstavlja funkcionalno povezanu realizaciju tri glavna dela projektnog zadatka, pri čemu PostgreSQL/PostGIS baza ima ulogu centralnog sloja koji povezuje relaciono upravljanje podacima, geoprostornu obradu, interaktivni kartografski prikaz i mašinsko učenje. OpenStreetMap podaci koriste se za formiranje početnog prostornog katastra, dok se rezultati YOLO detekcije čuvaju u istoj bazi i uključuju u tabelarne, kartografske i prostorne analize. Posebna vrednost rešenja ogleda se u tome što ML rezultati nisu predstavljeni samo kao anotirane fotografije, već kao prostorni zapisi koji mogu da se porede sa postojećim objektima i koriste kao kandidati za ručnu proveru i dopunu evidencije. Time je formirana osnova sistema koji objedinjuje evidenciju, analizu i podršku ažuriranju podataka o saobraćajnoj signalizaciji.

Literatura i izvori podataka

- PostgreSQL Global Development Group — PostgreSQL Documentation, <https://www.postgresql.org/docs/>
- PostGIS Project Steering Committee — PostGIS Manual, <https://postgis.net/documentation/>
- Geofabrik GmbH — OpenStreetMap Data Extracts, <https://download.geofabrik.de/>
- GeoPandas Developers — GeoPandas Documentation, <https://geopandas.org/>
- Shapely Contributors — Shapely Documentation, <https://shapely.readthedocs.io/>
- Ultralytics — YOLO Documentation, <https://docs.ultralytics.com/>
- Folium Contributors — Folium Documentation, <https://python-visualization.github.io/folium/>
- Streamlit Inc. — Streamlit Documentation, <https://docs.streamlit.io/>
- psycopg2 Contributors — psycopg2 Documentation, <https://www.psycopg.org/docs/>
- pandas Development Team — pandas Documentation, <https://pandas.pydata.org/docs/>